Week 0: Getting Ready

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

This unit is taught on a machine. From the first laboratory you will be reading permissions, looking at what runs as whom, and deciding whether something is a weakness. None of that is hard, but all of it assumes a floor: a few dozen commands and about six ideas. Some of you have that floor from a previous unit or a job. Others come from degrees where the command line was never the point, and there is nothing wrong with that.

This document gets everybody to the same starting line, in about ninety minutes, on the same virtual machine the unit uses. Everything here is optional and nothing is marked.

Contents

- Should you do this?
- Part 1: Get the machine running (about 30 minutes)
- Part 2: The tour (about 45 minutes)
- Self-check
- What Week 1 will ask
- If you got stuck
- Answers to the self-check

Should you do this?

Read the twelve statements below. If you can do **all twelve** on a Linux machine without looking anything up, skip this document: you already have the floor and Week 1 will not surprise you.

1. Tell what directory you are in, and move to another one by an absolute and a relative path.
2. Read `-rw-r-----` and say who may read the file, who may write it, and who may do neither.
3. Change a file's permissions with `chmod`, using either the letters or the three digits.
4. Say what is different between having write permission on a **file** and having it on the **directory** the file sits in.
5. Print your own user and group identities, and say what being in a group gets you.
6. Run a command with `sudo` and explain what that changes about who the process is.
7. List the processes running on the machine and say which user each one runs as.
8. Send the output of one command into another with a pipe, and save output to a file.
9. Find a line containing a word inside a file, and find a file by name under a directory.
10. Check whether a system service is running, and read the last few lines of its log.
11. List what is listening on a network port, and say which program and which account is behind each one.
12. Edit a file in a terminal editor, and read only the lines of a configuration file that are actually set.

If two or three of those are shaky, do the sections that cover them. If most of them are, do the whole thing in order. It is written to be done at the keyboard, not read.

Part 1: Get the machine running (about 30 minutes)

Follow the **Setup Wizard** published alongside this document. It covers both paths: the `.utm` bundle for an Apple Silicon Mac and the `.ova` for an Intel or AMD laptop. The bundle is pre-configured, so you do not build a virtual machine or change any hypervisor settings.

The wizard also shows you how to work on the machine from your own computer's terminal, over SSH. Do that from the start: the machine's own window has a US keyboard layout and no copy and paste with your computer, and your terminal has both.

When the machine boots you will see a normal `login:` prompt. Log in as `student`, password `student`. Then confirm the machine is the right one:

```
cat /etc/ssc-image-release
```

That should name the SSC lab image and your architecture. Post that line in the Week 0 forum thread on the course platform. It takes ten seconds and it tells the teaching team, before the first laboratory, whose machine boots. **Being stuck is the expected state at this point,** and that is what the thread is for.

The self-test

Then run this. It is one command, it checks everything the thirteen laboratories will need, and it names the laboratory beside anything that is missing. Paste the whole block.

```
bad=""
chk(){ if eval "$2" >/dev/null 2>&1; then printf '  ok       %s\n' "$1"; else printf
 ↪  '  MISSING  %s\n' "$1"; bad="$bad
  - $1"; fi; }
have(){ command -v "$1" >/dev/null 2>&1 || [ -x "/usr/sbin/$1" ] || [ -x "/sbin/$1"
 ↪  ]; }
note(){ if eval "$2" >/dev/null 2>&1; then printf '  ok       %s\n' "$1"; else
 ↪  printf '  later    %s\n' "$1"; fi; }
echo "SSC lab self-test"; echo
chk "this is the SSC image"  "[ -r /etc/ssc-image-release ]"
cat /etc/ssc-image-release 2>/dev/null
echo
chk "docker daemon reachable as student, every container lab"  "docker info"
note "alpine cached, Labs 9 to 12 (if 'later': docker pull alpine)"  "docker image
 ↪  inspect alpine"
chk "docker compose, Lab 11"                                  "docker compose version"
chk "gcc, Labs 7, 9 and 10"                                       "have gcc"
chk "jq and openssl, Lab 4"                                  "have jq && have openssl"
chk "ssh-keygen, Lab 5"                                           "have ssh-keygen"
chk "getcap and setcap, Lab 7"                                    "have getcap && have
 ↪  setcap"
chk "AppArmor tools, Lab 8"                                       "have aa-status &&
 ↪  have apparmor_parser"
chk "auditd, Lab 8"                                               "have auditctl"
chk "strace, Labs 8 and 10"                                       "have strace"
chk "unshare, Lab 9"                                              "have unshare"
chk "python3 seccomp bindings, Lab 10"                            "python3 -c 'import
 ↪  seccomp'"
chk "scmp_sys_resolver, Lab 10"                              "have scmp_sys_resolver"
echo
if [ -z "$bad" ]; then
  echo "READY. Everything the thirteen labs need is installed."
  echo "This cannot check your ssc-clean snapshot. Take it now, with the VM powered
   ↪  off."
  echo "That one is your safety net: the machine as it arrived. Each lab asks for
   ↪  its own."
else
  printf 'NOT READY. Missing:%s\n\n' "$bad"
  echo "Post this output in the Week 0 forum thread, before the first lab."
fi
```

The line you want is the last one:

```
READY. Everything the thirteen labs need is installed.
```

If it says `NOT READY`, the list above it names what is missing and which laboratory needs it. **Post that output in the forum thread rather than trying to fix it by guessing**: a missing package usually means an older bundle, and re-downloading is faster and safer than installing pieces by hand.

Two things this deliberately does not do. It reads nothing about this machine's own configuration, so it does not spoil the first laboratory, whose whole exercise is finding out what is on here. And it cannot check your snapshot, because nothing running inside a virtual machine can see whether one exists. That one is on you, and the Setup Wizard's last step tells you how.

Now make yourself a place to work. Everything in Part 2 happens inside it, and nothing in Part 2 touches the rest of the machine:

```
mkdir -p ~/week0 && cd ~/week0
```

Part 2: The tour (about 45 minutes)

Work in order. Every command is meant to be typed, and every step asks you to predict what will happen before you press Enter. Predicting and being wrong is how this sticks.

A. Where you are, and what is here

```
pwd                 # print working directory: where am I?
ls                  # what is here
ls -l               # the same, with detail
ls -la              # and including names that start with a dot
cd ..               # up one level
pwd
cd ~/week0          # back; ~ means your home directory
```

Two habits worth acquiring now. **Tab completion:** type `cd ~/we` and press Tab; the shell finishes the name. It is not a shortcut, it is how you avoid typing a wrong path. **The manual:** `man ls` opens the documentation for a command, and `q` leaves it. Every command in this unit has a manual page on the machine, and reading it there beats searching the web for an answer that may be about a different system.

B. Files, permissions and ownership

This is the section that matters most. Almost every finding in the first two laboratories is a permission that somebody set without thinking.

```
echo "hello" > notes.txt     # create a file, with one line in it
ls -l notes.txt
```

The first column looks like `-rw-rw-r--`. Read it in four pieces: the first character is the type (`-` for an ordinary file, `d` for a directory), then **three groups of three**: what the **owner** may do, what the **group** may do, and what **everybody else** may do. Each group is read, write, execute, in that order, with `-` where the permission is absent.

```
chmod 600 notes.txt         # owner may read and write; nobody else anything
ls -l notes.txt
chmod 644 notes.txt         # owner read and write; everyone else read only
ls -l notes.txt
```

The three digits are the same three groups, written as numbers: read is 4, write is 2, execute is 1, and you add them up. So 6 is read plus write, 4 is read only, 7 is everything. 644 therefore means owner 6, group 4, others 4.

Now the distinction that the second lecture is built on:

```
mkdir cage
echo "important" > cage/target.txt
chmod 444 cage/target.txt     # read-only for everybody, including you
ls -l cage/target.txt
echo "overwrite" > cage/target.txt   # predict: does this work?
```

That fails: you have no write permission on the file. Now try the other way in:

```
rm -f cage/target.txt         # predict: does this work?
ls -l cage
```

That **succeeds**, and it is not a bug. Removing a file is a change to the **directory** that contains it, not to the file. Your permission on `cage` is what decided it. Write access to a directory is the right to replace what sits in it, whatever the permissions on the contents say.

Remember that one. It comes back in Week 2 as an attack.

C. Who you are

```
whoami        # the short answer
id            # the full answer: user, primary group, and every group you are in
groups        # just the group names
```

You are `student`, and you are in the groups `sudo` and `docker`. Being in a group is not a title; it is a set of permissions that apply to you because files are owned by that group.

```
id -u         # your numeric user id
id -u root    # root is always 0
```

The system works in numbers and shows you names. `sudo` runs a single command as another user, `root` by default:

```
whoami
sudo whoami   # it will ask for your password: student
```

The two answers differ, and that difference is the whole subject of Week 6. A **service account** is a user that exists for a program rather than for a person: no password, no login, just an identity for something that runs. You will meet several in Week 1.

D. Processes

A process is a running program, and it carries the identity of whoever started it.

```
ps                    # your own processes in this terminal
ps aux | head -20     # every process on the machine, with its user in the first
 ↪  column
```

Start something that keeps running, look at it, then stop it:

```
sleep 300 &           # & means: run it in the background
ps aux | grep sleep   # find it; note the number in the second column, its PID
kill %1               # stop the background job you started
ps aux | grep sleep   # gone
```

The `USER` column in `ps aux` is the point of the exercise. When Week 1 asks you what a machine is running, it is asking for that column as much as for the program names.

E. Text, pipes and finding things

```
echo "alpha" > a.txt
echo "beta"  > b.txt
```

```
echo "alpha and beta" > c.txt

grep alpha a.txt          # is this word in this file?
grep -l alpha *.txt       # which files contain it?
grep -n alpha c.txt       # with line numbers
```

The **pipe** | sends the output of one command into the next, and > sends it into a file:

```
ls -l | grep txt          # only the lines mentioning txt
ls -l > listing.txt       # write it to a file instead of the screen
cat listing.txt
wc -l listing.txt         # how many lines
```

Finding files by name, inside your own directory:

```
find ~/week0 -name "*.txt"
find ~/week0 -name "*.txt" -newer a.txt
```

Note the quotes around `*.txt`. Without them the shell expands the pattern before `find` sees it, which is a good first example of why quoting matters.

F. Services and their logs

Most of what a Linux machine does is done by services started at boot. Look at one that is certainly running, the SSH server:

```
systemctl status ssh              # running or not, and since when
systemctl cat ssh                 # the unit file: what it runs, and as whom
sudo journalctl -u ssh -n 20      # the last twenty log lines for this service
```

In the middle command, find the `ExecStart=` line: that is the program the machine runs. The line above it, `ExecStartPre=`, is a check that runs first, so read past it. Now look for a `User=` line. **There is not one**, and that absence is the answer: with no `User=`, the service runs as `root`. A line that is missing is a decision as much as a line that is present, and reading what a file does *not* say is half of this unit.

Note that the log needed `sudo` while the other two did not. Reading service logs is privileged on this system: without it you get an empty answer rather than an error, which is a trap worth meeting now instead of in Week 8. Press `q` to leave any of these when it fills the screen.

G. The network, at a minimum

Start a small web server of your own, on a port nobody else is using, and talk to it:

```
cd ~/week0
python3 -m http.server 8099 &     # your own server, in the background
curl http://localhost:8099/       # ask it for the directory listing
ss -tlnp | grep 8099              # confirm something is listening on that port
kill %1                           # stop it
curl http://localhost:8099/       # predict: what happens now?
```

`ss -tlnp` lists what is listening on the machine, and it is the single most useful command in Week 1. Here you pointed it at your own server; next week you will point it at everything.

H. Editing a file, and reading configuration

```
nano notes.txt
```

Type a line, then press Ctrl+O and Enter to save, Ctrl+X to leave. If you prefer another editor, use it; nothing in this unit depends on which one.

Configuration on Linux is almost always plain text, usually with # starting a comment. Look at a real one, read-only:

```
grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'   # the lines that are actually set
```

One habit for later: some configuration files are **generated** by a tool and get overwritten. They say so at the top. Reading the first three lines of a file before editing it is a cheap way to avoid making a change that quietly disappears next month.

Finished

```
cd ~ && rm -rf ~/week0
```

That removes the sandbox. Nothing you did in this document changed anything outside it.

Self-check

Answer without going back. The answers are at the end.

1. What does `pwd` tell you?
2. In `-rw-r--r--`, who may write the file?
3. What does `chmod 640` set, in words?
4. A file is `444` and you own the directory it is in. Can you delete it?
5. Which command shows every group you belong to?
6. What changes about a process when it is started with `sudo`?
7. In `ps aux`, what is in the first column?
8. What does `ls -l | grep conf` do, in one sentence?
9. Which command shows what a service runs, and as which user?
10. Which command shows what is listening on a network port?
11. How do you read only the lines of a configuration file that are actually set, ignoring comments and blank lines?

What Week 1 will ask

The first laboratory asks you to enumerate the attack surface of this machine, across four dimensions: **what listens**, **what is privileged**, **what is scheduled**, and **what is exposed at rest**. Every command you need for that is in Part 2 above, pointed at the whole machine instead of at your sandbox.

Do not go looking now. Arriving with the machinery in your hands and the machine unexamined is exactly the right state to be in, and finding things for yourself is the point of the session.

If you got stuck

Post in the Week 0 thread, with the exact error and your operating system. Being stuck on an installation is the most common thing in the world, and somebody in the thread has usually met your problem already. Do it before the first laboratory: the session keeps some time for a machine that misbehaves, but a first download and installation does not fit in it.

Answers to the self-check

1. The directory you are currently in.
2. Only the owner. Group and others may read only.
3. Owner may read and write, group may read, others get nothing.
4. Yes. Deleting is a change to the directory, not to the file.

5. `id`, or `groups` for just the names.
6. It runs with another user's identity, `root` by default, so it is subject to that user's permissions rather than yours.
7. The user the process runs as.
8. Lists the files here in detail, and keeps only the lines whose text contains `conf`.
9. `systemctl cat <service>`: the `ExecStart=` line, and the `User=` line if there is one. No `User=` means it runs as `root`.
10. `ss -tlnp`.
11. Filter it. Part H uses `grep -v '^#' <file> | grep -v '^$'`, which drops the lines that begin with a # and the empty ones. A comment or a blank line that starts with a space or a tab survives that, so the form that answers the question as asked is `grep -vE '^[[:space:]]*(#|$)' <file>`. Either way, read what comes back rather than trusting the filter.